§2.4 线性表的链式表示———双链表、循环链表、静态链表

一、双链表定义与初始化

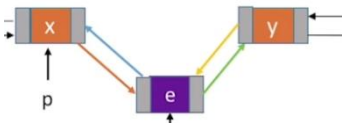
1. 定义

```
typedef struct DNode {
    ElemType data;          // 数据域
    struct DNode *prior;    // 前驱指针
    struct DNode *next;     // 后继指针
} DNode, *DLinklist;
```

2. 初始化（带头结点）

```
bool InitDLinklist(DLinklist *L) {
    L = (DNode *)malloc(sizeof(DNode));
    if (L == NULL)
        return false;
    L->prior = NULL;
    L->next = NULL;
    return true;
}

bool Empty(DLinklist L) {            //判断是否为空表
    return (L->next == NULL);
}
```



4. 插入操作（在 p 结点之后插入 s 结点）

本来由于双链表具有两个指针的特性，应该有四步。如果 p 是最后一点，②会指向 NULL。

```
bool InsertNextDNode(DNode *p, DNode *s) {

    ①s->next = p->next;

   ②p->next->prior = s;
   ③s->prior = p;
   ④p->next = s;
}
```

```
bool InsertNextDNode(DNode *p, DNode *s) {
if(p == NULL || s == NULL)   //非法参数
return false;
①s->next = p->next;
if(p ->next != NULL)            //如果 p 是最后一个结点，则没有指回 e 的 prior
②p->next->prior = s;
 ③ s->prior = p;
 ④ p->next = s;
return true;
}
```

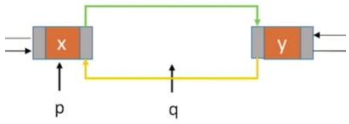
注意：

①要遵循“先后再前”的顺序，防止出 bug（单链表插入的逻辑完全相同）

②双链表不讨论前插，由于 prior 存在，可以把前插转化为前驱结点的后插。

5. 结点删除

```
//一般思路
p ->next = q ->next;
q ->next ->prior = p;
free(q)
```



如果 q 是最后一点，第二步会出错。下面是健壮代码

```
//删除p结点的后继结点
bool DeleteNextDNode(DNode *p){
    if (p==NULL)    return false;
    DNode *q = p->next;     //找到p的后继结点q
    if (q==NULL)    return false;   //p没有后继
    p->next=q->next;
    if (q->next!=NULL)      //q结点不是最后一个结点
        q->next->prior=p;
    free(q);                //释放结点空间
    return true;
}
```

把这个函数循环运转，就可以销毁链表：

```
void DestroyList(DLinklist &L) {
    while (L->next != NULL) {
        DeleteNextNode(L);          // 删除后继结点
    }
    free(L);
    L = NULL;
}
```

6. 遍历操作

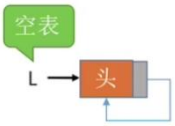
关键步骤： p = p->next、 p = p->prior

双链表不支持随机存取，查找操作需要遍历，时间复杂度为 O(n)。

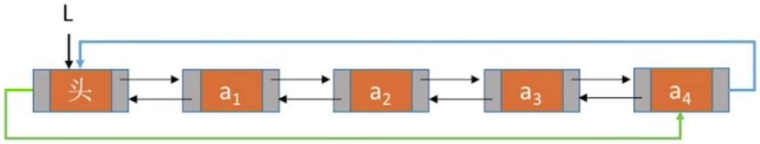
二、循环链表

1. 循环单链表（表尾结点的 next 指向头结点，形成循环）

- 初始化：L->next = L;
- ps：普通单链表的初始化 L->next = NULL;
- 判空：if(L->next == L)
- 判断 p 是否在表尾：if(p->next == L)



2. 循环双链表（表头 prior 指向表尾结点。表尾 next 指向头结点）

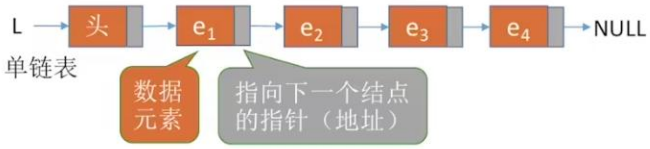


- 初始化：头结点的 prior、next 均指向头结点自己
- ps：普通双链表的初始化，头结点的 prior、next 指向 NULL；
- 判空：if(L->next == L)
- 判断 p 是否在表尾：if(p->next == L)
- 插入：在普通双链表的插入，即一.4 中左边的代码，②不够健壮。但如果是循环双链表，逻辑可以说得通。
- 删除：同样在一.5，原来的代码是可行的。

注意：循环双链表的判断操作可以用 prior 指针实现，但一般不这样做

三、静态链表（单链表在内存中离散存放，静态链表是在一片连续空间中存放）

VS



游标是数组下标，next 指针储存地址

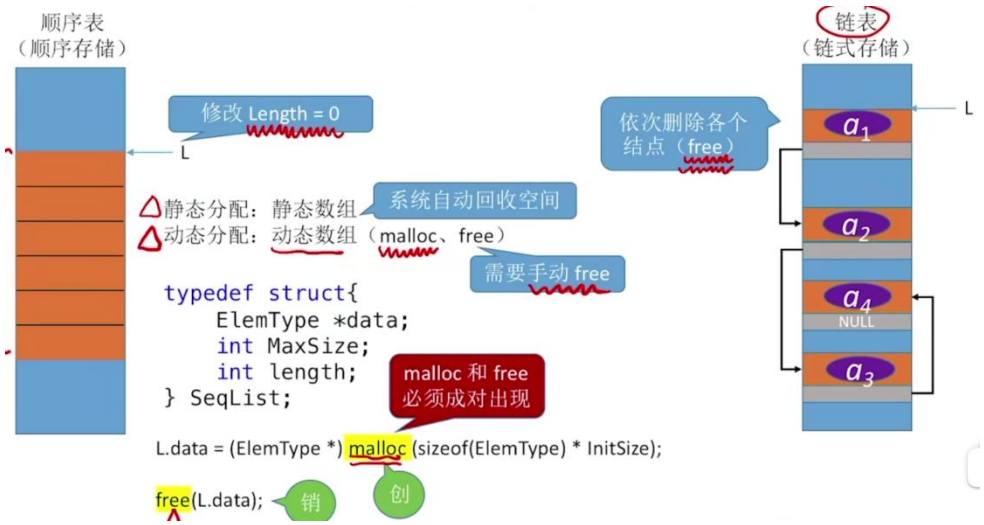
静态链表的地址：数组中第 i 个元素的地址=addr+i*8

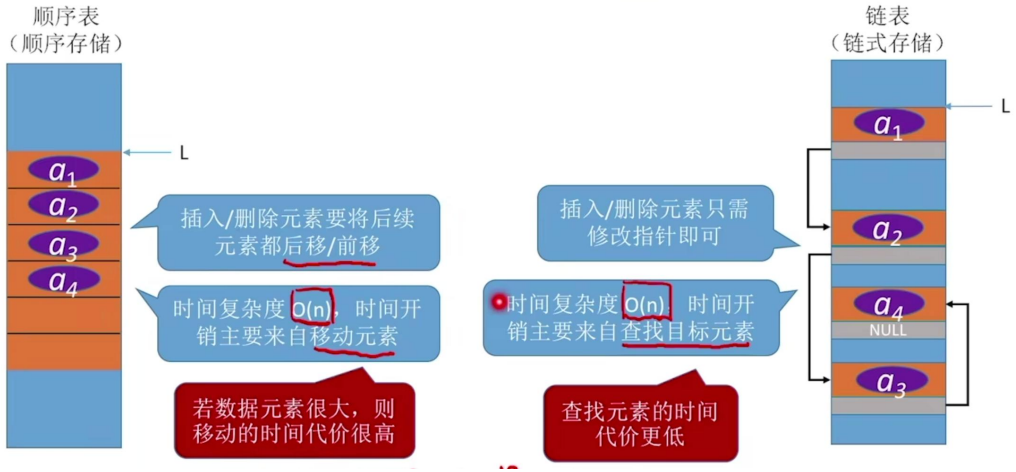
1. 静态链表定义

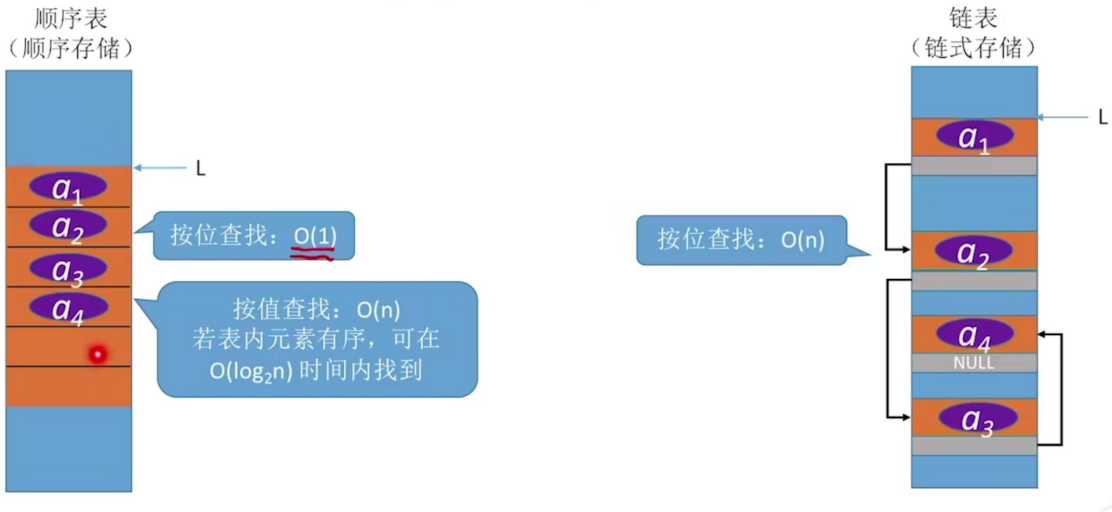
```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;        // 游标，指向下一个元素的下标
};
void testSLinkList(){
    struct Node a[MaxSize];          //数组 a[]作为静态链表
    ......
}
```

2.基本操作

- **初始化**：a[0] ->next = -1;
- **查找**：从头结点开始遍历游标







	顺序表	链表
弹性（可扩容）	😭	😄
增、删	😭	😄
查	😄	😭